

Bản trình chiếu: Thiết kế và tối ưu hàm băm

Li Yuxiu

2005

Nguồn gốc: Thiết kế và tối ưu hàm băm: bản trình chiếu

I0I China candidate team papers / 2005 / Li Yuxiu / Li Yuxiu.ppt

1 Mạch trình bày

PowerPoint gốc là phần tóm lược trực quan của bài viết *Thiết kế và tối ưu hàm băm*. Văn bản trích được từ slide xác nhận các mốc chính: băm số nguyên bằng phép chia $h(k) = k \bmod M$, lấy bit giữa của k^2 , phương pháp nhân với $A = 0.61803$, băm chuỗi như I0I2005, các hàm SDBMHash, MD5, SHA1, rồi phần “hashing” so với “numerize” cho hoán vị và chỉnh hợp.

Bản ghi chú này giữ nhịp của slide: trước hết hỏi thế nào là một hàm băm tốt, sau đó đi qua từng loại khóa thường gặp trong thi lập trình. Các đoạn chương trình trong bài đầy đủ không được chép lại ở đây; với code, chỉ nêu vai trò và công thức cập nhật đủ để người đọc nhận ra thuật toán.

2 Tiêu chí của một hàm băm

Mục tiêu của hàm băm là biến khóa k thành một chỉ số trong bảng, sao cho các khóa thật sự xuất hiện được phân bố càng đều càng tốt. Nếu nhiều khóa rơi vào cùng một ô, ta phải xử lý va chạm nhiều hơn, bộ nhớ bị dùng kém hiệu quả và các thao tác tra cứu mất thời gian.

Slide dùng các ví dụ rất nhỏ để nhấn mạnh rằng công thức nhìn đơn giản vẫn có thể bị dữ liệu có cấu trúc đánh bại. Vì thế khi thiết kế băm trong bài thi, ta không chỉ quan tâm tới tính đúng của phép ánh xạ vào $[0, M)$, mà còn cần xem khóa đầu vào có quy luật nào, bảng có kích thước nào, và phép tính có đủ nhanh so với số lần gọi hay không.

3 Băm số nguyên

Cách thông dụng nhất là lấy dư trực tiếp:

$$h(k) = k \bmod M.$$

Slide minh họa với $k = 100$, $M = 12$, nên $h(k) = 4$. Ưu điểm là cài đặt ngắn và tốc độ cao. Nhược điểm là chất lượng phụ thuộc mạnh vào M . Nếu M có quan hệ đặc biệt với dữ liệu, ví dụ nhiều khóa cùng chia hết cho một ước chung, các khóa sẽ dồn cụm. Bài viết đầy đủ khuyên chọn M tương đối “không đều”, thường là số nguyên tố hoặc ít nhất tránh các lũy thừa quá đặc biệt khi khóa có cấu trúc.

Một biến thể khác là lấy một số bit từ phần giữa của k^2 . Với $M = 2^r$, giá trị băm có thể hiểu là lấy r bit. Slide ghi ví dụ $r = 4$, $k = 100 = (1100100)_2$; sau khi bình phương và chọn phần giữa, thu được

$h(k) = (1000)_2 = 8$. Cách này dùng phép nhân và phép bit nên nhanh, nhưng không phải lúc nào cũng phân bố đều, đặc biệt với khóa nhỏ hoặc khóa liên tiếp.

Phương pháp nhân chọn một số thực $A \in (0, 1)$, lấy phần lẻ của kA , rồi nhân với kích thước bảng:

$$h(k) = \lfloor M\{kA\} \rfloor.$$

Slide dùng $A = 0.61803$, gần $(\sqrt{5} - 1)/2$, và $M = 12$. Phương pháp này ít bị phụ thuộc trực tiếp vào tính chia hết của M , nhưng đổi lại thường chậm hơn vì dùng số thực. Trong bối cảnh thi đấu, nó hữu ích khi phép lấy dư trực tiếp dễ gặp mẫu xấu, nhưng không nên xem là lựa chọn mặc định nếu tốc độ là nút thắt.

4 Băm chuỗi

Với chuỗi, một cách tự nhiên là xem chuỗi như một số trong cơ số 256, hoặc cơ số 128 nếu chỉ xét ký tự ASCII. Slide đưa ví dụ `str = I0I2005`, $M = 23$, và coi chuỗi là số hexa `0x494F4932303035`; khi lấy modulo `0x17`, giá trị băm là 13. Các chuỗi gần nhau như `N0I2005` và `I0I2004` cho các giá trị khác, minh họa ý tưởng chuyển dãy ký tự thành một khóa số.

Tuy nhiên, phép đổi cơ số đơn thuần vẫn có bẫy. Nếu chọn modulo không khéo, ví dụ có liên hệ với cơ số, hoán vị ký tự hoặc các chuỗi có tổng ký tự giống nhau có thể va chạm nhiều. Vì vậy các hàm băm chuỗi thực dụng thường trộn thêm phép dịch bit, phép cộng, phép trừ hoặc phép xor để mỗi ký tự ảnh hưởng tới nhiều bit của kết quả.

Slide so sánh băm trực tiếp với SDBMHash. Thay vì chép mã C, có thể tóm tắt lõi của SDBMHash bằng truy hồi

$$hash \leftarrow hash \cdot 65599 + ch$$

cho từng ký tự ch , sau đó lấy phần dương cần dùng. Các giá trị slide ghi cho `I0I2005`, `N0I2005` và `I0I2004` cho thấy chỉ một thay đổi nhỏ trong chuỗi đã làm giá trị cuối thay đổi mạnh, đúng với mục tiêu “khuếch tán” của hàm băm chuỗi.

MD5 và SHA1 cũng xuất hiện trên slide. Đây là hàm băm mật mã, thiết kế để khó tìm va chạm theo nghĩa an toàn thông tin. Trong thi lập trình thông thường, ta ít cần chi phí lớn như vậy; nhưng ví dụ này nhắc rằng khi dữ liệu có khả năng bị đối thủ chọn cố ý, độ bền trước va chạm có thể quan trọng hơn vài phép toán tiết kiệm được.

5 Từ băm tới đánh số trạng thái

Phần cuối của slide chuyển từ “hashing” sang “numerize”. Đây là điểm khác biệt quan trọng. Băm cho phép nhiều khóa cùng rơi vào một ô và phải xử lý va chạm. Còn đánh số trạng thái cố gắng tạo song ánh giữa một tập đối tượng và các số nguyên liên tiếp, từ đó có thể định địa chỉ trực tiếp.

Với hoán vị của $1, \dots, n$, bài viết dùng hệ cơ số giai thừa. Một số từ 0 tới $n! - 1$ có thể viết duy nhất dưới dạng

$$x = a_1 \cdot 1! + a_2 \cdot 2! + \dots + a_{n-1} \cdot (n-1)!, \quad 0 \leq a_i \leq i.$$

Dãy chữ số a_i tương ứng với mã Lehmer của hoán vị. Nhờ đó, mỗi hoán vị được gán một số duy nhất trong khoảng 1 tới $n!$, rất phù hợp cho tìm kiếm trạng thái, quy hoạch động nén trạng thái hoặc bảng đánh dấu đã thăm.

Slide cũng ghi công thức cho chỉnh hợp $A(n, m) = n(n-1) \dots (n-m+1)$. Ý tưởng tương tự: chữ số đầu có n lựa chọn, chữ số tiếp theo có $n-1$ lựa chọn, và tiếp tục giảm. Đây không còn là băm xác suất nữa, mà là mã hóa chính xác cấu trúc tổ hợp.

6 Tổng kết

Thông điệp của bản trình chiếu là thực dụng. Với số nguyên, lấy dư trực tiếp thường là lựa chọn đầu tiên, nhưng cần chọn M cẩn thận. Với chuỗi, nên dùng hàm trộn đã được kiểm nghiệm như SDBM hoặc JS thay vì chỉ cộng ký tự đơn giản. Với hoán vị và các cấu trúc tổ hợp nhỏ, nếu có thể đánh số song ánh thì nên định địa chỉ trực tiếp, vì khi đó ta tránh hoàn toàn vấn đề va chạm.

Một hàm băm tốt trong thi đấu không nhất thiết phải “đẹp” về mặt mã. Nó cần đủ đều với dữ liệu của bài, đủ nhanh trong số lần gọi dự kiến, và đủ đơn giản để cài chắc. Slide vì vậy không đưa ra một công thức vạn năng, mà cung cấp một hộp công cụ để chọn cách băm phù hợp với kiểu khóa cụ thể.